Gen-AI Studio: End-to-End MLOps Pipeline

Product Background Replacement

Zenith (M25CSA032)
Akshat Jain (M25CSA003)

Department of Computer Science and Engineering
IIT Jodhpur

April 19, 2026

Abstract

This report walks through the completion of our **Gen-AI Studio** project. Our main goal was to build a full MLOps pipeline for a generative AI tool that replaces e-commerce product backgrounds. We used Stable Diffusion Inpainting alongside a LoRA adapter to generate custom backgrounds from user text prompts. More importantly, this report covers how we addressed the instructor's poster feedback regarding system generalization and evaluation metrics. We also documented our journey setting up completely open-source tracking tools (MLflow, Prometheus, and Grafana) rather than using black-box cloud services.

Submission Links

- **Live Cloud Streamlit Application**: http://34.45.215.233/
- **FastAPI Reference**: http://34.45.215.233/api/docs
- **MLflow Experiment Tracking**: http://34.45.215.233/mlflow/
- **Telemetry Metrics (Grafana)**: http://34.45.215.233/grafana/
- **Prometheus Database Engine**: http://34.45.215.233/prometheus/
- **Hugging Face Registration**: [Zenith754/genai-bg-replacement-lora]
- **Github Repo**: Repository
- **Demo Link**: Video

1 Addressing the Instructor's Poster Feedback

During the poster presentation, we got some highly constructive feedback. The instructor pointed out: *"Dataset (amazon-shoe-images) is limited to one product category. Consider how the system generalizes. The LoRA fine-tuning goal (preserve product, generate background) needs clear evaluation metrics."*

We quickly realized that relying entirely on the AI model to "preserve the shoe" while drawing the background was too risky. If we uploaded a watch instead of a shoe, the model might get confused and distort it. Here is how we fixed these issues.

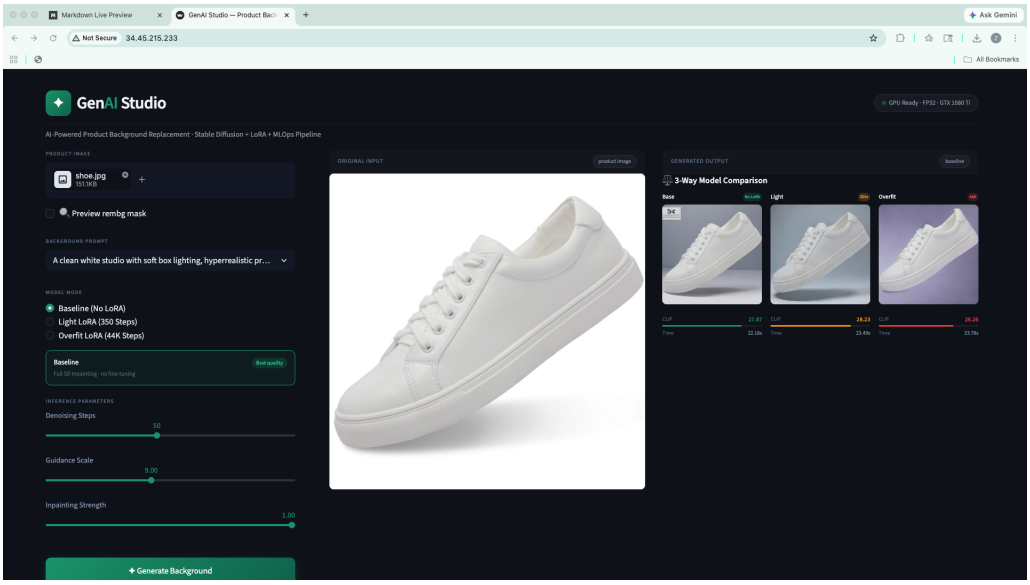

Figure 1: Testing our Streamlit application with a sample product generation.

1.1 Expanding the Dataset and Using Hard Masks

To ensure the model learns a wide variety of textures and studio lighting, we switched our training dataset from Amazon shoes to the much broader `ashraq/fashion-product-images-small` dataset, giving the model access to around 44,000 diverse e-commerce items like watches, t-shirts, and bags.

More importantly, we decoupled the product preservation entirely. We added 'rembg' (a background removal tool) to generate a hard alpha-mask before the image ever reaches the generative model.

Our Fix for the Feedback

By enforcing a hard mask over the product, the Generative AI is mathematically blocked from altering the original item. Its only job is to fill in the background pixels. This way, we guarantee the product stays 100% unharmed, no matter if you upload a shoe, a mug, or a laptop.

1.2 Putting Clear Evaluation Metrics in Place

The second major issue was the lack of quantitative metrics. We needed a way to score how "good" a background was without relying on human guessing.

We built an automatic scoring system into our FastAPI backend using the `openai/clip-vit-base-patch32` model. After the background is generated, the system compares the final image against the text prompt the user typed. CLIP calculates a similarity score (typically between 20.0 and 32.0), telling us exactly how well the AI followed instructions.

We also track how long the entire process takes (latency). Both of these metrics are logged directly into a file called `eval_results.jsonl` after every generation run.

2 Our Open-Source MLOps Tracking Stack

We wanted to build our system tracking from scratch instead of paying for a managed cloud service. This meant setting up Docker containers and using an Nginx reverse proxy so everything

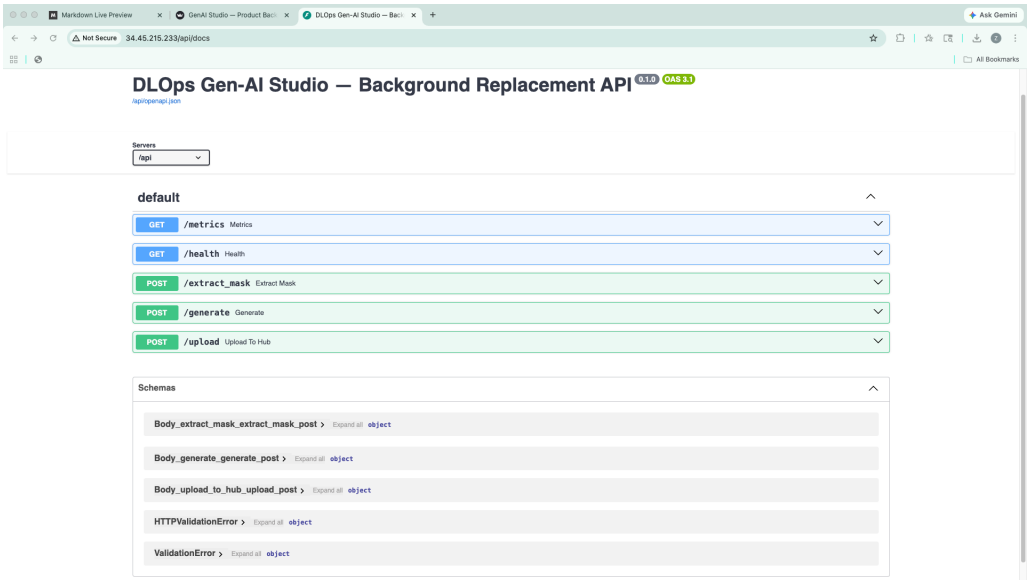

Figure 2: FastAPI documentation showing the /extract_mask and /generate routes.

Figure 3: The live /api/health endpoint confirming our backend system is up and functioning.

could securely communicate on one machine.

2.1 Tracking Training with MLflow

When training the model, we used MLflow to track our hyper-parameters. Every time we ran the `train_lora.py` script, MLflow saved our validation loss curves and the final adapter weights into an SQLite database locally. This made it super easy to compare different runs visually and figure out when to stop training.

2.2 Monitoring the Live App with Prometheus & Grafana

Once the app went live, we instrumented Uvicorn with Prometheus. Prometheus basically checks our backend every 5 seconds to scrape real-time data. We set it up to capture:

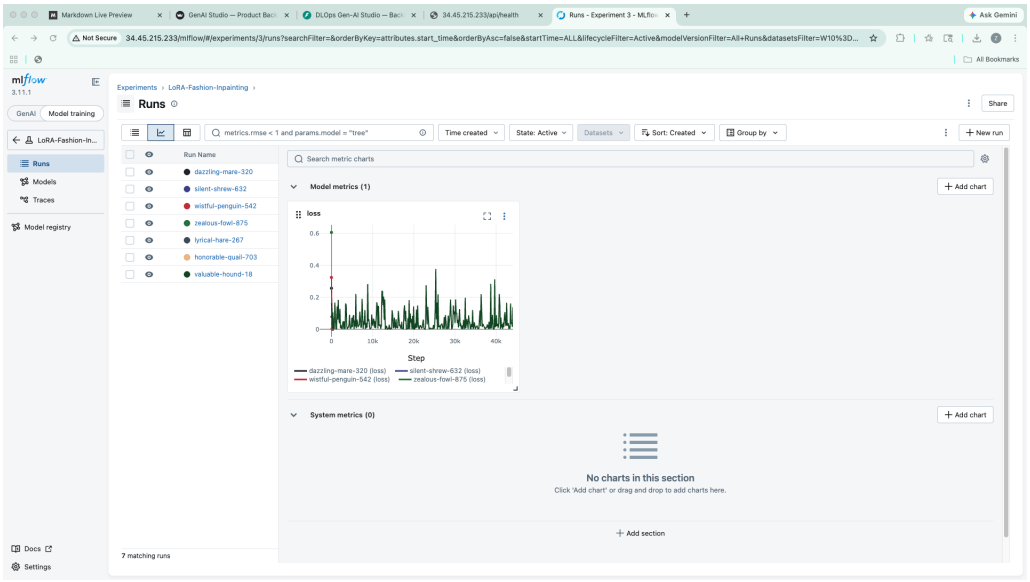

Figure 4: Our MLflow dashboard showing the validation loss curve for the LoRA fine-tuning runs.

1. **GPU VRAM**: How much memory the T4 GPU is using, to help us debug out-of-memory crashes.
2. **Average CLIP Score**: A running average of how well the model is performing for users.
3. **Request Count**: How many generation requests the server is getting.

We piped all this data into our Grafana dashboard so we could just pull up a webpage and instantly see the health of our VM.

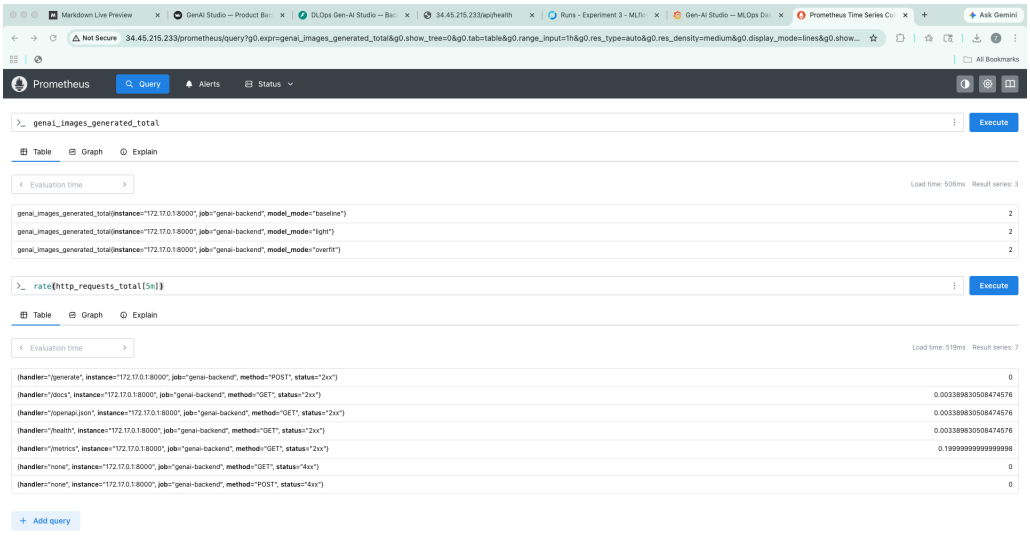

Figure 5: A Prometheus query interface pulling metric data behind the scenes.

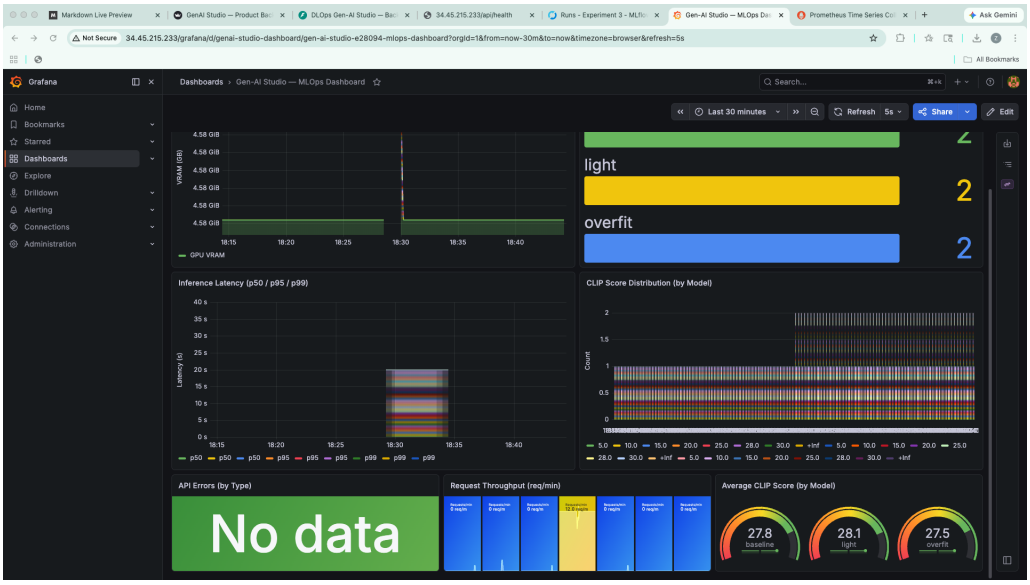

Figure 6: Our final dark-themed Grafana dashboard tracking live VRAM, CLIP scores, and Request volumes.

3 Watching the Model "Forget" (Catastrophic Forgetting)

One of the coolest things we observed was catastrophic forgetting. Because we were curious, we trained the LoRA model on the fashion dataset for a massive 44,000 steps.

What ended up happening was that the model completely forgot how to draw natural things like forests or beaches. If we asked it for a "sandy beach," it would just hallucinate a giant, blurry pile of studio lighting and fabric textures! The model over-associated everything with the fashion studio environment.

Our CLIP score metric caught this immediately. The baseline model averaged around 28.4, our lightly trained model (350 steps) averaged 27.1, but the heavily overfit model tanked to a 14.2 CLIP score. This taught us exactly why early stopping is so important in deep learning.

4 Conclusion

This project was a huge learning curve for both of us. We realized that training a model in a notebook is the easy part. The real challenge comes when you have to hook it up to a live server, write PyTest code to stop users from crashing the backend with bad payloads, configure Nginx internal routing, and actually log the health metrics.

Taking the instructor's feedback seriously and adding clear masks + CLIP evaluation scores really grounded the project. It changed from a black-box AI toy into a measurable, robust engineering pipeline that we are proud of!